ASP.NET Core Server & Hosting Architecture

---

1. Core Concept — Hosting in ASP.NET Core

Every ASP.NET Core application requires a **web server** to receive HTTP requests and return HTTP responses. ASP.NET Core supports **two primary hosting models**:

1. **In-Process Hosting**
2. **Out-of-Process Hosting (Reverse Proxy Model)**

The difference lies in **where the application runs and which process executes the request pipeline**.

---

2. In-Process Hosting Model (IIS as the Web Server and Host Process)

2.1 Definition

In the **In-Process hosting model**, the ASP.NET Core application executes **inside the IIS worker process**. This means that IIS itself hosts the runtime and the application.

Worker processes involved:

Server	Process Name
IIS	w3wp.exe
IIS Express	iisexpress.exe

2.2 Configuration

Configured in the project file:

<AspNetCoreHostingModel>InProcess</AspNetCoreHostingModel>

This is the **default when deploying to IIS on Windows**.

2.3 Request Execution Flow

The flow is:

Client → IIS → ASP.NET Core Application (same process)

There is:
✓ No proxy
✓ No secondary web server
✓ No inter-process network hop

2.4 Why In-Process Hosting Is Faster

Performance is higher because:

- There is **only one server involved**
- The request **never leaves IIS**
- No HTTP forwarding occurs
- No Kestrel routing is required

Result:
✓ Lower latency
✓ Higher throughput

2.5 Platform Support

In-Process hosting works **only on Windows**, since IIS is a Windows-only web server.

2.6 Summary Statements

- IIS is the **only web server handling the request**
- Application runs inside **w3wp.exe / iisexpress.exe**
- **Kestrel is not used at all**
- Performance is optimal due to **no reverse proxy overhead**

---

3. Out-of-Process Hosting Model (Reverse Proxy with Kestrel)

3.1 Definition

In **Out-of-Process hosting**, the ASP.NET Core application runs in its own **separate Kestrel process**, while IIS / Apache / Nginx works as a **reverse proxy server**.

3.2 Configuration

<AspNetCoreHostingModel>OutOfProcess</AspNetCoreHostingModel>

3.3 Request Execution Flow

The request flow becomes:

Client → IIS / Apache / Nginx → Kestrel → Application

Two web servers exist:

1. **External reverse proxy server**
2. **Internal Kestrel server**

3.4 Why a Reverse Proxy Is Required

A reverse proxy provides:

- Security & traffic protection
- SSL termination
- Request filtering
- Windows authentication
- Load balancing capacity
- Smart URL routing
- Edge protection

Kestrel alone is **not hardened enough for direct internet exposure** in many enterprise scenarios, therefore most production deployments use a proxy.

3.5 Processes Running

Internal execution occurs inside:

- dotnet.exe
- or YourProjectName.exe

while the proxy runs under the web server process such as w3wp.exe.

3.6 Platform Support

Out-of-Process hosting works on:

✓ Windows
✓ Linux
✓ macOS

because Kestrel is cross-platform.

---

4. Comparison — In-Process vs Out-of-Process

Feature	In-Process	Out-of-Process
Internal Server	IIS	Kestrel
External Server	None	IIS / Apache / Nginx
Worker Process	w3wp.exe / iisexpress.exe	dotnet.exe
Performance	Faster	Slightly slower due to proxy hop
CLI Execution	Not used	Always used
Platform	Windows only	Cross-platform

---

5. Kestrel Web Server

5.1 Definition

Kestrel is the **default internal web server** for ASP.NET Core. It is:

✓ Cross-platform
✓ High-performance
✓ Lightweight
✓ Required for Out-of-Process hosting

Process names:

- dotnet.exe
- YourProjectName.exe

5.2 Two Operating Modes of Kestrel

Mode 1 — Stand-Alone Mode

Request Flow:

Client → Kestrel

Occurs when:

dotnet run

or when using the **Project profile** from Visual Studio.

Mode 2 — Reverse Proxy Mode

Request Flow:

Client → IIS / Apache / Nginx → Kestrel

This is the **most common production configuration**.

---

6. Running ASP.NET Core via CLI

6.1 Command

dotnet run

6.2 Key Facts

✓ Always uses **Kestrel**
✓ **Ignores HostingModel setting**
✓ Always executes **Out-of-Process**
✓ Runs inside **dotnet.exe**

This means **CLI execution does not use IIS at all**.

---

7. Can ASP.NET Core Run Without Kestrel?

✓ Yes — but **only in In-Process hosting mode**

In that scenario:

- IIS hosts the application directly
- Application executes inside **w3wp.exe / iisexpress.exe**
- **Kestrel is not started**

This is the **only scenario where Kestrel is not used**.

---

8. launchSettings.json — Development Hosting Profiles

8.1 Location

Properties/launchSettings.json

Used **only in development**, not in production.

8.2 Profiles and Server Mapping

commandName	Server Used
Project	Kestrel
IISExpress	IIS Express
IIS	IIS

8.3 Execution Rules

How Application Is Run	Profile Used	Server Used
Run from Visual Studio (Default)	IISExpress	IIS Express
Run using dotnet run	Project	Kestrel

So:

- Visual Studio default → IIS Express
- CLI dotnet run → always Kestrel
